Семинар 2. Введение в многопоточное программирование

Принципы построения высоконагруженных систем

Георгий Семенов

Институт прикладных компьютерных наук, Университет ИТМО

9 октября 2026

Почему мы будем говорить о многопоточном программировании?

- На практике, **высоконагруженная система** – набор экземпляров сервисов, соединенных сетью и обслуживающих динамическую нагрузку
- Каждый **экземпляр сервиса** исполняется в своем изолированном окружении (например, в Docker-контейнере внутри пода Kubernetes)
- Чем выше *пропускная способность* (throughput) одного экземпляра сервиса, тем меньше их требуется для поддержания одной и той же нагрузки
- Одно из ключевых средств оптимизации – использование параллелизма процессора и **многопоточности** исполняемой среды (рантайма языка, виртуальной машины, операционной системы и т.д.)

План семинара

1. Что такое операционная система?
2. Понятия программы, процесса и потока
3. Многопоточное исполнение
    - модель, порядок выполнения и аномалии в памяти
    - примитивы синхронизации, барьеры, атомики, mutex
4. Где многопоточность в высоконагруженном бэкенде?
    - Java: `ThreadPool`, Netty
5. Concurrency: корутины, virtual threads и отличие от "простых" потоков

1. Что такое операционная система?

ОС — это **слой управления ресурсами**, который даёт программам безопасный и стандартизованный доступ к железу (CPU, RAM, Storage, I/O).



> **Ключевая идея:** пользовательский код не работает напрямую с железом; он вызывает ОС, а ОС делегирует выполнение и контролирует доступ к ресурсам.

1. ОС и системные вызовы

```
Приложение
    ├─ open("data.txt")
    ├─ read(...)
    ├─ write(...)
    └─ socket(), connect(), send()
          ↓
       ядро ОС
          ↓
       CPU / RAM / Disk / NIC
```

ОС: изоляция процессов и памяти, планирование потоков на CPU, файловую систему и сетевой стек, синхронизацию доступа к аппаратуре, управление временем и ресурсами.

> Без ОС программа будет просто набором инструкций; ОС превращает это в управляемую среду выполнения.

2. Программа, процесс, поток

Программа

- статический код
- executable-файл, библиотека, скрипт
- лежит на диске и не исполняется сам по себе

Процесс

- запущенный экземпляр программы
- владеет адресным пространством, файлами, дескрипторами, окружением
- имеет свой контекст выполнения

Поток

- единица выполнения внутри процесса
- делит память процесса, но имеет собственный стек и регистры
- много потоков внутри процесса исполняются параллельно или конкурируют за CPU

2. Отношение между понятиями

Агрегация / композиция

- **Program** → может быть запущен как один или несколько **Process**
- **Process** → содержит набор **Thread**
- **Thread** → выполняет инструкции в общей памяти процесса

```
Program (код на диске)
        ↓
Process (запущенный экземпляр)
        ├── Thread 1
        ├── Thread 2
        └── Thread N
```

Важный момент

Несколько потоков в одном процессе не являются независимыми программами: они **делят одно и то же адресное пространство**.

Это даёт производительность, но вводит проблему **shared mutable state**.

3. Многопоточное исполнение

Потоки нужны, когда:

- есть несколько независимых задач
- можно распараллелить вычисления
- нужны фоновые workers / обработчики I/O
- хочется добиться высокой пропускной способности и отзывчивости

Но это не бесплатный выигрыш: чем больше потоков, тем больше:

- contention за ресурсы,
- lock contention,
- cache thrashing,
- риск race conditions.

3. Порядок команд не определён

```
int ready = 0;
int data = 0;

// Thread A
void writer() {
    data = 42;
    ready = 1;
}

// Thread B
void reader() {
    if (ready == 1) {
        printf("%d\n", data);
    }
}
```

Что может случиться?

Что может случиться?

Поток B может увидеть:

- `ready == 1` и `data == 0`
- `ready == 1` и `data == 42`
- либо вообще другую заметную последовательность в зависимости от архитектуры, компилятора и планировщика

> Это не "ошибка программиста" в духе опечатки. Это нормально для многопоточной модели памяти: порядок наблюдения не обязателен.

3. Memory model и reordering

Компилятор и процессор могут менять порядок инструкций, если это не нарушает *логическую корректность* для одного потока.

Пример:

```
A: x = 1
B: y = 2
```

может быть выполнено как

```
B: y = 2
A: x = 1
```

если для текущего потока это "эквивалентно".

Но между потоками это создаёт видимость, которая не совпадает с буквальным исходным кодом:

- отсутствие синхронизации ⇒ отсутствие гарантии на наблюдаемый порядок
- для корректности нужен явный контракт синхронизации

3. Примитивы синхронизации

1) Барьеры (`fence` / memory barrier)

- запрещают переупорядочивание операций
- обеспечивают видимость записей между потоками

2) Атомики

- операции типа `fetch_add`, `compare_exchange`, `store` выполняются как единое целое
- часто используются для счётчиков, флагов, очередей

3) Mutex / Lock

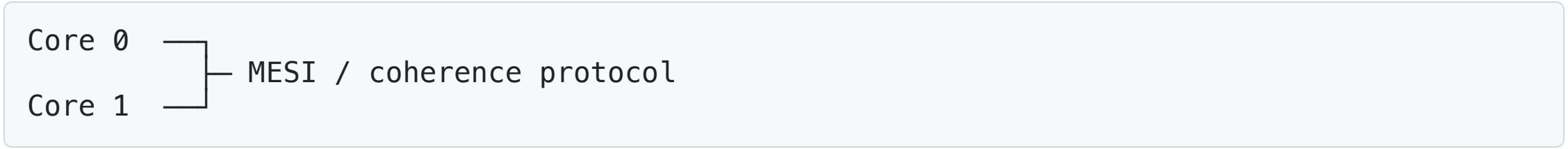
- сериализует критическую секцию
- гарантирует взаимоисключение доступа к shared state

4) RW lock, semaphore, condition variable

- регулируют чтение/запись, очереди ожидания и координацию потоков

3. Кэш и когерентность

У современных CPU есть несколько уровней кэша:



Распространённый протокол — **MESI**: Modified, Exclusive, Shared, Invalid.

Что это значит?

Когда один поток изменяет переменную, другие ядра должны получить уведомление, что строка кэша стала невалидной.

Иначе:

- один поток видит старое значение,
- другой — новое,
- возникает ложное чувство "параллельности" без корректной согласованности.

Барьеры и атомики как раз помогают сделать эту согласованность видимой и предсказуемой.

3. Lock-free и wait-free

Lock-free

- хотя бы один поток завершает операцию в конечное время
- не обязательно, что все потоки будут прогрессировать

Wait-free

- каждый поток гарантированно завершит операцию за конечное число шагов
- это более сильное условие

Примеры

- lock-free queue
- ring buffer with atomic head/tail
- atomic counters / refcounts
- Treiber stack

Но есть нюанс

4. Где многопоточность в высоконагруженном бэкенде: `ThreadPool`?

Основа высоконагруженного Java-бэкенда — обычно работа с несколькими слоями: входящие соединения, чтение сокетов, парсинг протокола, бизнес-логика, запись ответов

```
ExecutorService pool = new ThreadPoolExecutor(
    8,
    32,
    60L,
    TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(1000),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

Что тут важно?

- `corePoolSize` — базовый размер пула
- `maxPoolSize` — верхняя граница
- `queue` — буфер задач
- `CallerRunsPolicy` — защита от бесконечного роста очереди

4. Netty: event loop

Netty строит модель вокруг **event loop**:

```
Boss threads          Worker threads
    │                     │
    ├─ accept()           ├─ read()
    ├─ register()         ├─ decode()
    │                     ├─ handler()
    │                     └─ write()
```

Основная идея

- один loop обслуживает много соединений
- не создаётся отдельный поток на каждый TCP-сокет
- задачи распределяются по event loop'ам

Почему это важно?

- меньше накладных расходов на потоки
- лучше масштабирование на миллионы соединений

4. Конфигурирование и практика

В Java-проекте обычно спрашивают:

- сколько `core threads` надо выделить?
- какой размер очереди допустим?
- что делать при переполнении?
- как ограничить конкуренцию за CPU?
- что делать с blocking I/O?

Практические правила

- I/O-bound задачи: экзекутор с умеренным числом потоков, но аккуратной очередью
- CPU-bound задачи: не увеличивать число потоков бесконечно, иначе контекстные переключения будут мешать
- если external call блокирует поток — лучше разделить I/O и compute

> **Основной вопрос:** не “сколько потоков добавлять”, а “на сколько задач реально распараллеливается работа и где bottleneck”.

5. C++: userver

`userver` — хороший пример архитектуры, где работа идёт через **задачи / task-ы / корутины/пулы**, а не через “один поток на один запрос”.

Посмотрим отдельно презентацию! :)

5. Kotlin: coroutines

```
launch(Dispatchers.IO) {
    val data = fetchFromDb()
    val result = process(data)
    println(result)
}
```

Посмотрим отдельно презентацию! :)

Итог

- ОС — это среда, которая управляет CPU, RAM, I/O и памятью.
- Программа, процесс и поток — разные уровни абстракции.
- Многопоточность делает систему параллельной, но привносит **memory-ordering**, **race conditions** и **contention**.
- Синхронизация нужна не “по привычке”, а чтобы задать корректный контракт видимости и порядка.
- В высоконагруженном бэкенде обычно используют **pools, event loops, coroutines и virtual threads** в зависимости от смысла задачи.

> Вопрос для финала: не “сколько потоков?”, а “какой уровень concurrency реально нужен и где находится bottleneck?”